Linux Commands - Practice Exercises

Level 1: BEGINNER EXERCISES

Exercise 1.1: File Navigation and Listing

Create this directory structure and practice navigation:

```bash
# Create the structure
mkdir -p ~/practice/documents/2024/project
mkdir -p ~/practice/pictures/vacation
mkdir -p ~/practice/code/python
mkdir -p ~/practice/code/javascript

# Practice tasks:
1. Navigate to ~/practice
2. List all files with details (ls -l)
3. Go to documents/2024 directory
4. Print current directory (pwd)
5. Go back to parent directory
6. List hidden files
7. Go to code/python using relative path
8. Go to practice directory using absolute path
```

Solution:

```bash
cd ~/practice
ls -l
cd documents/2024
pwd
cd ..
ls -la
cd ../code/python
cd ~/practice
```

Exercise 1.2: Creating and Editing Files

Create and modify files:

```bash
```

```
# Tasks:
1. Create a file named "notes.txt"
2. Add content using echo: "My Learning Journal"
3. Create multiple files: task1.txt, task2.txt, task3.txt
4. View content of notes.txt
5. Add more content to notes.txt using nano
6. Save and exit
```

Solution:

bash

```
touch notes.txt
echo "My Learning Journal" > notes.txt
touch task1.txt task2.txt task3.txt
cat notes.txt
nano notes.txt
# In nano: type content, Ctrl+X, Y, Enter
```

Exercise 1.3: File Operations

Practice copying, moving, and deleting:

bash

```
# Setup
mkdir -p ~/practice/backup
mkdir -p ~/practice/archive

# Tasks:
1. Create a file: ~/practice/document.txt
2. Copy it to ~/practice/backup/
3. Move document.txt to ~/practice/archive/
4. List all files in archive directory
5. Copy everything from archive to backup
```

Solution:

bash

```
touch ~/practice/document.txt
cp ~/practice/document.txt ~/practice/backup/
mv ~/practice/archive/document.txt ~/practice/archive/
ls -l ~/practice/archive/
cp ~/practice/archive/* ~/practice/backup/
```

Level 2: INTERMEDIATE EXERCISES

Exercise 2.1: Grep and Text Searching

Create sample data and search it:

```bash
# Create sample log file
cat > ~/practice/app.log << 'EOF'
2024-03-20 10:15:23 ERROR Database connection failed
2024-03-20 10:16:45 WARNING Low memory available
2024-03-20 10:17:12 INFO Application started
2024-03-20 10:18:33 ERROR Timeout while fetching data
2024-03-20 10:19:50 INFO User login successful
2024-03-20 10:20:15 ERROR Database connection failed
2024-03-20 10:21:30 DEBUG Cache cleared
2024-03-20 10:22:45 ERROR Authentication failed
EOF

# Tasks:
1. Find all ERROR lines
2. Count how many ERROR lines exist
3. Find lines with ERROR but not "connection"
4. Show line numbers of ERROR lines
5. Find all lines and show 2 lines after each ERROR
6. Case-insensitive search for "warning"
7. Count total WARNING and ERROR lines
8. Get errors by hour (first task, shows creativity)
```

Solution:

```bash
grep "ERROR" ~/practice/app.log
grep -c "ERROR" ~/practice/app.log
grep "ERROR" ~/practice/app.log | grep -v "connection"
grep -n "ERROR" ~/practice/app.log
grep -A 2 "ERROR" ~/practice/app.log
grep -i "warning" ~/practice/app.log
grep -E "ERROR|WARNING" ~/practice/app.log | wc -l
grep "ERROR" ~/practice/app.log | awk '{print $1, $2}' | cut -d: -f1 | sort | uniq -
```

Practice changing file permissions:

```bash
# Create test files
cd ~/practice
touch script.sh config.conf secret.key

# Tasks:
1. Make script.sh executable for owner only
2. Make config.conf readable by everyone but writable only by owner
3. Make secret.key readable/writable by owner only
4. Verify permissions with ls -l
5. Change all files to 755 (owner: full, group: r-x, others: r-x)
6. Create directory with 750 permissions (owner: full, group: r-x, others: none)
7. Change nested directory permissions recursively
```

Solution:

```bash
chmod 700 script.sh
chmod 644 config.conf
chmod 600 secret.key
ls -l
chmod 755 script.sh config.conf secret.key
mkdir -p test_dir && chmod 750 test_dir
chmod -R 755 ~/practice/code
```

Exercise 2.3: Pipelines and Redirection

Combine commands using pipes:

```bash
```

```
# Create sample data
cat > ~/practice/users.txt << 'EOF'
john 1000
jane 2000
bob 1500
alice 3000
charlie 1800
EOF

# Tasks:
1. Sort by username
2. Sort by salary (second column)
3. Find users with salary > 1500
4. Count how many users
5. Show only usernames with salary > 1500
6. Create output file with sorted users
7. Append text to existing file
8. Show users in reverse alphabetical order
```

Solution:

bash

```
sort ~/practice/users.txt
sort -k2 -n ~/practice/users.txt
awk '$2 > 1500 {print}' ~/practice/users.txt
wc -l ~/practice/users.txt
awk '$2 > 1500 {print $1}' ~/practice/users.txt
sort ~/practice/users.txt > ~/practice/users_sorted.txt
echo "new_user 2500" >> ~/practice/users.txt
sort -r ~/practice/users.txt
```

Exercise 2.4: System Information

Check system status:

bash

```
# Tasks:
1. Show current user and user ID
2. List all running processes
3. Find Java processes only
4. Check disk usage in human-readable format
5. Find largest directory in home
6. Check memory usage
7. List listening ports
8. Show kernel version
```

Solution:

bash

```bash
whoami
ps aux
ps aux | grep java
df -h
du -sh ~/* | sort -rh | head
free -h
netstat -tulpn
uname -a
```

Level 3: ADVANCED EXERCISES

Exercise 3.1: Complex Log Analysis

Analyze a realistic application log:

bash

```
# Create realistic log file
cat > ~/practice/access.log << 'EOF'
192.168.1.100 - - [20/Mar/2024:10:00:05 +0000] "GET /index.html HTTP/1.1" 200 1234
192.168.1.101 - - [20/Mar/2024:10:00:10 +0000] "GET /style.css HTTP/1.1" 200 5678
192.168.1.100 - - [20/Mar/2024:10:00:15 +0000] "POST /api/login HTTP/1.1" 401 456
192.168.1.102 - - [20/Mar/2024:10:00:20 +0000] "GET /admin HTTP/1.1" 403 789
192.168.1.100 - - [20/Mar/2024:10:00:25 +0000] "GET /api/data HTTP/1.1" 200 9012
192.168.1.103 - - [20/Mar/2024:10:00:30 +0000] "GET /script.js HTTP/1.1" 404 123
192.168.1.101 - - [20/Mar/2024:10:00:35 +0000] "POST /api/submit HTTP/1.1" 500 345
192.168.1.100 - - [20/Mar/2024:10:00:40 +0000] "GET /api/data HTTP/1.1" 200 8901
EOF

# Tasks:
1. Count total requests
2. Find all failed requests (4xx, 5xx)
3. Count requests by IP address
4. Find most requested endpoints
5. Extract unique status codes
6. List all error responses with their status codes
7. Find requests that took error response by status
8. Create a summary report
```

Solution:

bash

```
wc -l ~/practice/access.log

grep " [45][0-9][0-9] " ~/practice/access.log

awk '{print $1}' ~/practice/access.log | sort | uniq -c | sort -rn

awk '{print $7}' ~/practice/access.log | sort | uniq -c | sort -rn

awk '{print $9}' ~/practice/access.log | sort -u

grep " [45][0-9][0-9] " ~/practice/access.log | awk '{print $9, $7}'

# Generate summary report
cat > ~/practice/log_summary.txt << 'REPORT'
Total Requests: $(wc -l < ~/practice/access.log)
Failed Requests: $(grep " [45][0-9][0-9] " ~/practice/access.log | wc -l)
REPORT
```

Exercise 3.2: Symbolic Links and Backups

Create a backup system using links:

```bash
# Create source files
mkdir -p ~/practice/source/project
echo "config data" > ~/practice/source/project/config.txt
echo "user data" > ~/practice/source/project/users.txt

# Tasks:
1. Create backup directory
2. Create symbolic links to source files in backup
3. Verify links work correctly
4. Show link information with ls -l
5. What happens if you delete original file?
6. Restore from backup by copying content
7. Create hard link backup (alternative method)
8. Compare size of symbolic vs hard links
```

Solution:

```bash
mkdir -p ~/practice/backup

ln -s ~/practice/source/project/config.txt ~/practice/backup/config.txt
ln -s ~/practice/source/project/users.txt ~/practice/backup/users.txt

cat ~/practice/backup/config.txt

ls -l ~/practice/backup/

# Hard link backup (takes same space)
ln ~/practice/source/project/config.txt ~/practice/backup/config.txt.hard

# Compare
ls -li ~/practice/backup/
```

Exercise 3.3: Find and Execute Complex Operations

Use find with advanced features:

```bash
```

```
# Setup
mkdir -p ~/practice/files
touch ~/practice/files/test.txt ~/practice/files/old.log
touch ~/practice/files/data.csv

# Tasks:
1. Find all .txt files
2. Find all files modified in last 7 days
3. Find files larger than 1KB
4. Find and delete all .log files
5. Find files and display in detail
6. Find files and execute multiple commands
7. Find empty files
8. Find and compress files
```

Solution:

bash

```
find ~/practice/files -name "*.txt"
find ~/practice/files -mtime -7
find ~/practice/files -size +1k
find ~/practice/files -name "*.log" -delete
find ~/practice/files -type f -exec ls -lh {} \;
find ~/practice/files -type f -size 0
find ~/practice/files -type f -name "*.log" -exec gzip {} \;
```

Exercise 3.4: System Monitoring Task

Monitor and analyze system resources:

bash

```
# Tasks:
1. List top 5 memory-consuming processes
2. Find CPU hog processes
3. Monitor disk usage and find largest directories
4. Check which ports are listening
5. Track network connections
6. Create a monitoring script
7. Schedule it to run every minute
8. Generate report from monitoring
```

Solution:

```bash
ps aux --sort=-%mem | head -6

ps aux --sort=-%cpu | head -6

du -sh /* | sort -rh | head -5

netstat -tulpn

netstat -an | grep ESTABLISHED | wc -l

# Create monitoring script
cat > ~/practice/monitor.sh << 'EOF'
#!/bin/bash
echo "=== System Monitoring Report ===" >> ~/practice/monitor.log
echo "Date: $(date)" >> ~/practice/monitor.log
echo "Memory Usage:" >> ~/practice/monitor.log
free -h >> ~/practice/monitor.log
echo "Disk Usage:" >> ~/practice/monitor.log
df -h >> ~/practice/monitor.log
echo "Top Processes:" >> ~/practice/monitor.log
ps aux --sort=-%mem | head -3 >> ~/practice/monitor.log
EOF

chmod +x ~/practice/monitor.sh
```

Level 4: REAL-WORLD PROJECTS

Project 1: Web Server Log Analysis

Scenario: You manage a web server and need to analyze traffic

```bash
```

```
# Create sample web server log
cat > ~/practice/web_traffic.log << 'EOF'
192.168.1.100 "GET /index.html" 200
192.168.1.101 "POST /api/login" 401
192.168.1.102 "GET /products" 200
192.168.1.100 "GET /styles.css" 200
192.168.1.103 "GET /admin" 403
192.168.1.101 "GET /contact" 200
192.168.1.100 "POST /api/submit" 500
192.168.1.102 "GET /blog" 200
192.168.1.104 "GET /index.html" 200
192.168.1.100 "GET /styles.css" 200
EOF

# Analysis tasks:
1. How many unique visitors?
2. Which IPs had errors?
3. Most visited endpoints?
4. Error rate percentage?
5. Suspicious activity (multiple errors from one IP)?
6. Peak usage time/pattern?
7. Generate summary report
8. Archive logs by date
```

Project 2: System Backup Solution

Scenario: Create an automated backup system

bash

```
# Requirements:
1. Create backup directories by date
2. Copy project files
3. Compress backup
4. Keep only last 7 days
5. Send email notification
6. Create restore script
7. Document the process
8. Schedule with cron
```

Implementation:

bash

```
# Create backup script
cat > ~/practice/backup_project.sh << 'EOF'
#!/bin/bash
BACKUP_DIR="/mnt/backup"
SOURCE_DIR="/app/project"
DATE=$(date +%Y%m%d_%H%M%S)

mkdir -p $BACKUP_DIR
cp -r $SOURCE_DIR $BACKUP_DIR/backup_$DATE
tar -czf $BACKUP_DIR/backup_$DATE.tar.gz $BACKUP_DIR/backup_$DATE
rm -rf $BACKUP_DIR/backup_$DATE

echo "Backup completed: backup_$DATE.tar.gz"
EOF

chmod +x ~/practice/backup_project.sh
```

Project 3: Configuration Management

Scenario: Manage multiple application configurations

bash

```
# Tasks:
1. Create base configuration
2. Create environment-specific configs
3. Use symbolic links to activate config
4. Create backup of previous configs
5. Switch between configurations
6. Version control configurations
7. Document changes
8. Automate deployment
```

BONUS CHALLENGES

Challenge 1: One-Liner Text Processing

Process a file in a single command:

bash

```
# Given: Large CSV file with user data
# Challenge: Extract users with score > 80, sort by score, show top 10

# Command:
awk -F',' '$3 > 80 {print}' data.csv | sort -t',' -k3 -nr | head -10
```

Challenge 2: System Health Check

Create a single command that shows system health:

bash

```
# Show: CPU, Memory, Disk, Network status
echo "=== System Health ===" && \
echo "CPU Usage: $(top -bn1 | grep load)" && \
echo "Memory: $(free -h | grep Mem)" && \
echo "Disk: $(df -h / | tail -1)" && \
echo "Network: $(netstat -an | grep ESTABLISHED | wc -l) connections"
```

Challenge 3: Find Pattern in Files

Search for specific pattern across entire project:

bash

```
# Find all TODO comments in code
find . -name "*.py" -o -name "*.js" | xargs grep -n "TODO\|FIXME\|BUG"
```

ANSWER KEY & EXPLANATIONS

Why each answer works:

1. **ls -l vs ls**: Shows detailed information including permissions and ownership
2. **grep -r**: Recursive search finds patterns in all files
3. **chmod 755**: Owner full control, others can read and execute (good for public scripts)
4. **ln -s**: Creates symbolic link (like Windows shortcut)
5. **| (pipe)**: Passes output of first command as input to second

---

TIPS FOR PRACTICE

1. **Create a dedicated practice directory**

```bash
mkdir -p ~/linux_practice
cd ~/linux_practice
```

2. **Start simple, then increase complexity**
   - Level 1: File operations
   - Level 2: Text processing
   - Level 3: System administration
   - Level 4: Real-world scenarios
3. **Always use -i flag for safety**

```bash
rm -i file.txt  # Asks before deleting
```

4. **Keep backup copies**

```bash
cp original.txt original.txt.backup
```

5. **Use man pages to learn**

```bash
man grep
man find
```

6. **Write down what you learn**
   - Create your own cheat sheet
   - Document solutions
   - Explain commands to others

EVALUATION CRITERIA

Level 1: Beginner (Weeks 1-2)

- [ ] Navigate file system confidently
- [ ] Create and edit files
- [ ] Understand basic ls options

- [ ] Copy, move, delete files safely
- [ ] Use nano editor effectively

Level 2: Intermediate (Weeks 3-4)

- [ ] Use grep for text searching
- [ ] Change file permissions correctly
- [ ] Understand symbolic links
- [ ] Use pipes and redirection
- [ ] Monitor system resources

Level 3: Advanced (Weeks 5-6)

- [ ] Complex text processing with awk/sed
- [ ] Create and manage backups
- [ ] Write shell scripts
- [ ] Automate repetitive tasks
- [ ] Analyze logs effectively

Level 4: Expert (Weeks 7+)

- [ ] Solve real-world problems
- [ ] Design system solutions
- [ ] Optimize performance
- [ ] Secure file systems
- [ ] Mentor others

RESOURCES FOR FURTHER LEARNING

1. **Online Tutorials**
   - Linux man pages online
   - StackOverflow for specific questions
   - YouTube for visual demonstrations
2. **Books**
   - "The Linux Command Line" by William E. Shotts Jr.
   - "Linux Pocket Guide" by Daniel J. Barrett
3. **Practice Platforms**
   - HackerRank Linux Challenges
   - LeetCode Shell Problems
   - Codewars Bash Exercises
4. **Professional Certifications**
   - CompTIA Linux+

- Red Hat Certified System Administrator (RHCSA)

Good luck with your practice! Remember: **The best way to learn is by doing!**